

STRUCTS

Prof^a Me. Monique Emídio de Oliveira

O QUE SÃO STRUCTS?

Em C, uma struct (estrutura) é um recurso que permite agrupar diferentes tipos de dados dentro de uma única variável.

Ou seja: enquanto uma variável comum armazena apenas um valor de um tipo, a struct pode armazenar várias informações relacionadas.

Exemplo da vida real:

Se quisermos representar um aluno, precisamos guardar:

- nome
- idade
- nota

Essas informações são diferentes entre si, mas pertencem ao mesmo aluno. Nesse caso, faz muito sentido usar uma struct.

SINTAXE BÁSICA DA STRUCT

```
struct NomeDaStruct {  
    tipo campo1;  
    tipo campo2;  
    tipo campo3;  
};
```

SINTAXE BÁSICA DA STRUCT

Exemplo:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

Nesse exemplo, criamos uma estrutura chamada Aluno, que possui:

- nome : vetor de caracteres (string)
- idade : inteiro
- nota : número decimal

SINTAXE BÁSICA DA STRUCT

- Depois de criar a estrutura, podemos declarar variáveis desse tipo.

struct Aluno a1;

- Agora a1 é uma variável do tipo struct Aluno.

Acessando os campos de uma struct

Usamos o operador ponto (.) para acessar os campos.

Exemplo:

```
a1.idade = 20;
```

```
a1.nota = 8.5;
```

Exemplo completo – Cadastro simples de aluno

```
#include <stdio.h>
#include <string.h>

struct Aluno {
    char nome[50];
    int idade;
    float nota;
};

int main() {
    struct Aluno a1;

    strcpy(a1.nome, "Maria");
    a1.idade = 19;
    a1.nota = 9.2;

    printf("Nome: %s\n", a1.nome);
    printf("Idade: %d\n", a1.idade);
    printf("Nota: %.2f\n", a1.nota);

    return 0;
}
```

Lendo dados do usuário com struct

Também podemos preencher os dados com `scanf` e `fgets`.

Lendo dados do usuário com struct

```
#include <stdio.h>
```

```
struct Pessoa {  
    char nome[50];  
    int idade;  
};
```

```
int main() {  
    struct Pessoa p;
```

```
    printf("Digite o nome: ");  
    scanf("%[^\n]", p.nome);
```

```
    printf("Digite a idade: ");  
    scanf("%d", &p.idade);
```

```
    printf("\n--- DADOS INFORMADOS ---\n");  
    printf("Nome: %s\n", p.nome);  
    printf("Idade: %d\n", p.idade);
```

```
    return 0;  
}
```

STRUCT COM TYPEDEF

Em C, é muito comum usar typedef com struct para facilitar a declaração.

Sem typedef:

```
struct Aluno a1;
```

Com typedef:

```
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;
```

Agora podemos declarar assim:

```
Aluno a1;
```

EXEMPLO COMPLETO

```
#include <stdio.h>
```

```
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;
```

```
int main() {  
    Aluno a1;
```

```
    printf("Digite o nome: ");  
    scanf(" %[^\\n]", a1.nome);
```

```
    printf("Digite a idade: ");  
    scanf("%d", &a1.idade);
```

```
    printf("Digite a nota: ");  
    scanf("%f", &a1.nota);
```

```
    printf("\\n--- DADOS DO ALUNO ---\\n");  
    printf("Nome: %s\\n", a1.nome);  
    printf("Idade: %d\\n", a1.idade);  
    printf("Nota: %.2f\\n", a1.nota);
```

```
    return 0;
```

```
}
```

VETOR DE STRUCTS: PARTE 1

```
#include <stdio.h>
```

```
typedef struct {  
    char nome[50];  
    float nota;  
} Aluno;
```

```
int main() {  
    Aluno turma[3];  
    int i;
```

```
    for(i = 0; i < 3; i++) {  
        printf("\nAluno %d\n", i + 1);
```

```
        printf("Digite o nome: ");  
        scanf(" %[^\\n]", turma[i].nome);
```

```
        printf("Digite a nota: ");  
        scanf("%f", &turma[i].nota);  
    }
```

VETOR DE STRUCTS: PARTE 2

```
printf("\n--- LISTA DE ALUNOS ---\n");  
    for(i = 0; i < 3; i++) {  
        printf("Aluno %d: %s - Nota: %.2f\n", i + 1, turma[i].nome,  
turma[i].nota);  
    }  
  
    return 0;  
}
```

Struct dentro de struct (struct aninhada)

- Uma struct pode conter outra struct dentro dela. Observe o código:

```
#include <stdio.h>
```

```
typedef struct {  
    char rua[50];  
    int numero;  
} Endereco;
```

```
typedef struct {  
    char nome[50];  
    int idade;  
    Endereco end;  
} Pessoa;
```

```
int main() {  
    Pessoa p;
```

```
    printf("Digite o nome: ");  
    scanf("%[^\n]", p.nome);
```

```
    printf("Digite a idade: ");  
    scanf("%d", &p.idade);
```

Continuação

```
printf("Digite a rua: ");  
scanf(" %[^\\n]", p.end.rua);
```

```
printf("Digite o numero: ");  
scanf("%d", &p.end.numero);
```

```
printf("\\n--- DADOS COMPLETOS ---\\n");  
printf("Nome: %s\\n", p.nome);  
printf("Idade: %d\\n", p.idade);  
printf("Rua: %s\\n", p.end.rua);  
printf("Numero: %d\\n", p.end.numero);
```

```
return 0;  
}
```

PASSANDO STRUCT PARA FUNÇÃO

Uma struct pode ser passada para uma função como parâmetro.

```
#include <stdio.h>
```

```
typedef struct {  
    char nome[50];  
    float preco;  
} Produto;
```

```
void exibirProduto(Produto p) {  
    printf("\n--- PRODUTO ---\n");  
    printf("Nome: %s\n", p.nome);  
    printf("Preco: R$ %.2f\n", p.preco);  
}
```

Continuação

```
int main() {  
    Produto prod;  
  
    printf("Digite o nome do produto: ");  
    scanf("%[^\n]", prod.nome);  
  
    printf("Digite o preco: ");  
    scanf("%f", &prod.preco);  
  
    exibirProduto(prod);  
  
    return 0;  
}
```

Passando struct por ponteiro

- Quando usamos ponteiro para struct, acessamos os campos com o operador:

```
#include <stdio.h>
```

```
typedef struct {  
    char nome[50];  
    int idade;  
} Pessoa;
```

```
void alterarIdade(Pessoa *p) {  
    p->idade = 30;  
}
```

```
int main() {  
    Pessoa pessoa1;
```

```
    printf("Digite o nome: ");  
    scanf(" %[^\\n]", pessoa1.nome);
```

CONTINUAÇÃO

```
printf("Digite a idade: ");  
scanf("%d", &pessoa1.idade);
```

```
alterarIdade(&pessoa1);
```

```
printf("\n--- DADOS ALTERADOS ---\n");  
printf("Nome: %s\n", pessoa1.nome);  
printf("Idade: %d\n", pessoa1.idade);
```

```
return 0;  
}
```

DIFERENÇA ENTRE . E ->

- . : usado quando temos a variável normal da struct
- -> : usado quando temos um ponteiro para struct

Exemplo:

```
Pessoa p;
```

```
Pessoa *ptr = &p;
```

```
p.idade = 20;
```

```
ptr->idade = 25;
```

•

Lista de exercícios no moodle